

Processos em Sistemas Distribuídos

Prof. Paulo Bressan

Universidade Federal de Alfenas — UNIFAL-MG

Sumário

Seção 1	Threads	Motivação, chaveamento de contexto, IPC, implementação e threads em sistemas distribuídos
Seção 2	Virtualização e Clientes	Tipos e arquiteturas de VM, interfaces de rede, X Window, thin clients e software do lado cliente
Seção 3	Servidores e Clusters	Tipos de servidor, estado, clusters, comutadores de camada de transporte e gerenciamento
Seção 4	Migração de Código	Motivação, modelos de mobilidade e vinculação recurso-máquina

Threads

A thick, hand-drawn style orange horizontal line that spans across the width of the slide, positioned below the title.

Motivação, chaveamento de contexto, IPC, implementação e threads em sistemas distribuídos

Introdução e contexto

- Em sistemas distribuídos de longa distância, mover processos e código entre máquinas ajuda na escalabilidade e no balanceamento de carga
- Configuração dinâmica de clientes e servidores: código que se adapta ao ambiente em tempo de execução (ex.: applets, agentes móveis)
- Essa aula trata das unidades de execução — processos e threads — que tornam essa movimentação e cooperação possíveis

Ferramentas / comandos

```
$ rsync -av origem/ destino/
```

Transferência eficiente de estado/código entre máquinas

```
$ scp arquivo usuario@host:/dest
```

Cópia simples via SSH

```
$ ansible-playbook deploy.yml
```

Reconfiguração remota automatizada

Motivação para threads

- Granularidade fina de controle por processo: múltiplos fluxos de execução compartilhando o mesmo espaço de endereçamento
- **Desempenho:** criar uma thread é ordens de magnitude mais barato que `fork()` de processo — não há cópia de tabela de páginas nem novo espaço de endereçamento
- Threads compartilham memória, arquivos abertos e descritores; cada uma mantém pilha, registradores e contador de programa próprios

Ferramentas / comandos

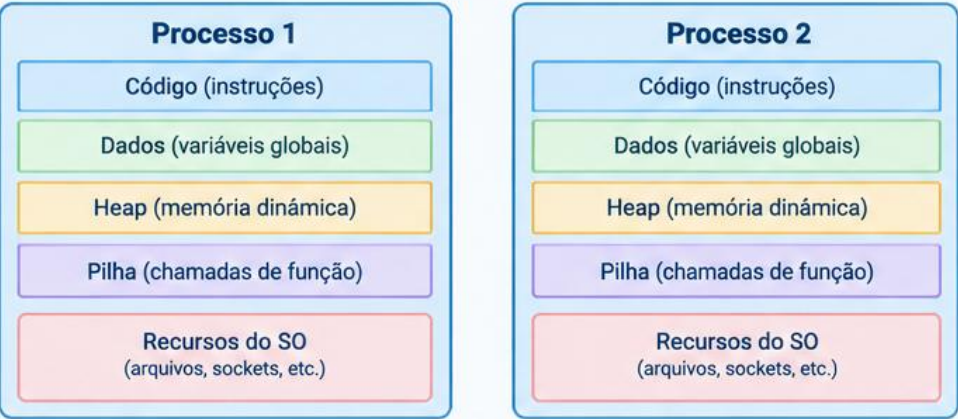
```
$ strace -f -e trace=clone,fork,execve  
./prog
```

Compara o custo de syscalls entre criar processo e criar thread

Threads e Processos: principais diferenças

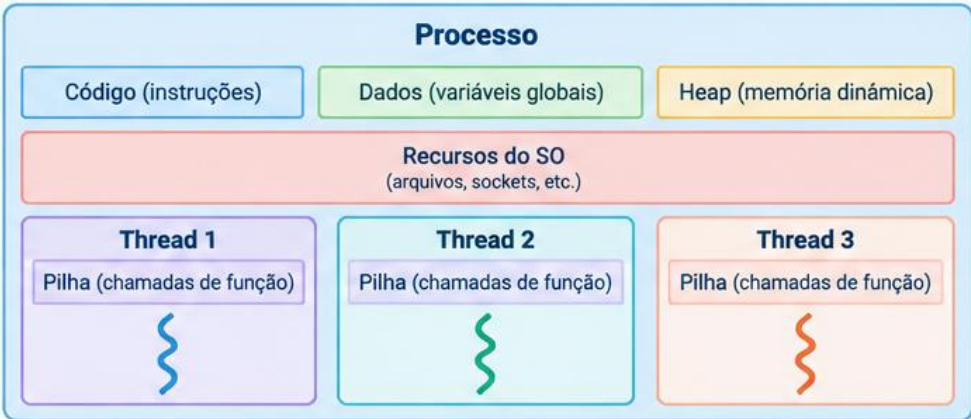
Processos

Um processo é um programa em execução, com seu próprio espaço de endereçamento e recursos.



Threads

Uma thread é uma unidade de execução dentro de um processo, compartilhando o mesmo espaço de endereçamento e recursos.



Aspecto	Processos	Threads
 Espaço de endereçamento	Cada processo tem seu próprio espaço de endereçamento (geralmente isolado).	Threads do mesmo processo compartilham o mesmo espaço de endereçamento.
 Recursos do sistema	Cada processo possui seus próprios recursos (arquivos, sockets, etc.).	Threads compartilham os recursos do processo (arquivos, sockets, etc.).
 Pilha	Cada processo tem sua própria pilha.	Cada thread tem sua própria pilha.
 Criação e finalização	Mais pesado e mais lento.	Mais leve e mais rápido.
 Comunicação	Mais complexa (ex.: IPC – memória compartilhada, pipes).	Mais simples (memória compartilhada, variáveis globais).
 Isolamento e falhas	Falhas em um processo geralmente não afetam outros.	Falhas em uma thread podem afetar todo o processo.
 Uso de CPU	Cada processo pode ser executado em um ou mais núcleos.	Várias threads de um mesmo processo podem ser executadas em paralelo em múltiplos núcleos.



Resumo:

Processos são independentes e oferecem isolamento, enquanto threads são mais leves e permitem maior compartilhamento de recursos e comunicação eficiente dentro de um mesmo processo.

Processadores virtuais e Chaveamento de Contexto

Em sistemas operacionais, um **processador virtual** é a abstração que o SO cria para dar a cada processo a ilusão de ter a CPU só para si, mesmo em uma máquina com poucos núcleos físicos rodando dezenas de processos "ao mesmo tempo".

Como funciona na prática

O SO mantém uma **tabela de processos**, que guarda o estado de cada processo (registradores, contador de programa, ponteiro de pilha, mapeamento de memória, etc.). Para dar a ilusão de multitarefa, o SO alterna rapidamente a CPU física entre os processos — e essa troca é o famoso **chaveamento de contexto**.

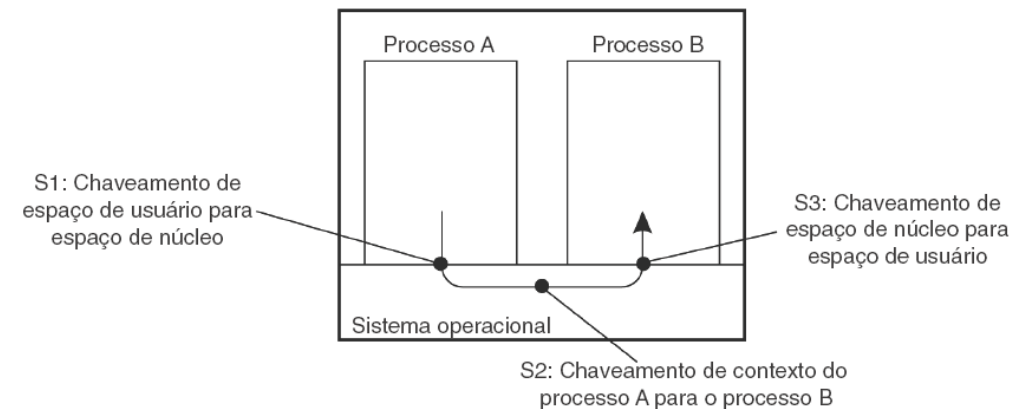


Figura 3.1 Chaveamento de contexto como resultado de IPC.

Processadores virtuais e Chaveamento de Contexto

Cada vez que a CPU é trocada de um processo para outro, o SO precisa:

1. **Salvar o contexto** do processo que está saindo — registradores de propósito geral, contador de programa (PC), ponteiro de pilha (SP)
2. **Atualizar os registradores da MMU** (Memory Management Unit) — tabelas de página, registradores de base/limite — para apontar para o espaço de endereçamento do novo processo
3. **Invalidar a cache TLB** (Translation Lookaside Buffer), já que ela guardava traduções de endereço válidas apenas para o processo anterior
4. **Restaurar o contexto** do processo que está entrando

Ferramentas / comandos

```
$ vmstat 1
```

Coluna cs: trocas de contexto por segundo

```
$ perf stat -e context-switches,cpu-migrations ./prog
```

Mede o custo real no seu programa

```
$ cat /proc/[pid]/status
```

voluntary/nonvoluntary_ctxt_switches

Processadores virtuais e Chaveamento de Contexto

Por que isso custa caro

Esse processo inteiro tem um custo real de desempenho — não é instantâneo.

Depois de invalidar a TLB, as primeiras instruções do processo "novo" sofrem *TLB misses*, exigindo que o SO refaça as traduções de endereço a partir das tabelas de página na memória, o que é bem mais lento que um acesso à TLB.

Em cenários de memória sob pressão, ainda pode haver troca dinâmica entre memória e disco (*swapping/paging*), que encarece ainda mais essa transição.

Por que isso importa para threads

É justamente esse custo que motiva o uso de **threads** em vez de múltiplos processos: como threads do mesmo processo compartilham o mesmo espaço de endereçamento, trocar entre elas não exige atualizar a MMU nem invalidar a TLB — só troca registradores, PC e pilha. Por isso a troca de contexto entre threads é ordens de magnitude mais barata que entre processos.

Ferramentas / comandos

```
$ vmstat 1
```

Coluna cs: trocas de contexto por segundo

```
$ perf stat -e context-switches,cpu-migrations ./prog
```

Mede o custo real no seu programa

```
$ cat /proc/[pid]/status
```

voluntary/nonvoluntary_ctxt_switches

Threads em Sistemas Distribuídos



Meio conveniente de permitir chamadas bloqueadoras de sistema sem bloquear o processo inteiro.



Expressar comunicação na forma de manter múltiplas conexões lógicas ao mesmo tempo.

Ferramentas / comandos

```
$ netstat -tnp
```

Conexões TCP simultâneas de um processo servidor

```
$ ss -tnp
```

Equivalente moderno, mais rápido

Threads em Sistemas Distribuídos

1. Eficiência e Desempenho

Menor custo de criação e gerenciamento:

- Threads são muito mais leves que processos — consomem menos memória e têm tempo de criação/destruição reduzido.

Troca de contexto rápida:

- Como compartilham o mesmo espaço de endereçamento, mudar o contexto entre threads é mais eficiente do que entre processos.

Melhor utilização da CPU:

- Threads podem executar em paralelo em múltiplos núcleos, explorando o **paralelismo de hardware**.

Threads em Sistemas Distribuídos

2. Comunicação Simplificada

Compartilhamento de memória:

- Threads de um mesmo processo podem se comunicar diretamente, sem precisar de mecanismos de IPC (como sockets ou filas).

Baixa latência de comunicação:

- Ideal para componentes que trocam mensagens frequentemente, como em servidores concorrentes.

Threads em Sistemas Distribuídos

3. Concorrência e Escalabilidade

Suporte a múltiplos clientes:

- Em servidores distribuídos (ex: servidores web, RPC, microserviços), cada cliente pode ser atendido por uma thread separada.

Escalabilidade horizontal facilitada:

- Aplicações *multi-threaded* podem ser replicadas e distribuídas em múltiplos nós com menos sobrecarga de sincronização.

Paralelismo natural:

- Threads permitem dividir uma tarefa em subtarefas executadas em paralelo (ex: tratamento de requisições, processamento de dados, replicação).

Threads em Sistemas Distribuídos

4. Reatividade e Responsividade

Maior responsividade:

- Enquanto uma thread aguarda E/S de rede ou disco, outras podem continuar executando — ideal para aplicações interativas e servidores de alta disponibilidade.

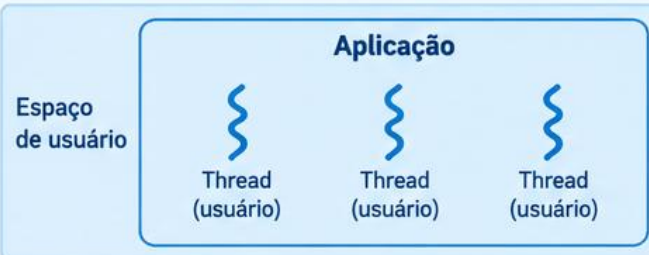
Tolerância parcial a falhas:

- Em alguns casos, falhas em uma thread podem ser tratadas sem comprometer todo o processo (quando há isolamento controlado).

Implementação de Threads: User-Level, Kernel-Level e Many-to-Many

User-Level Threads

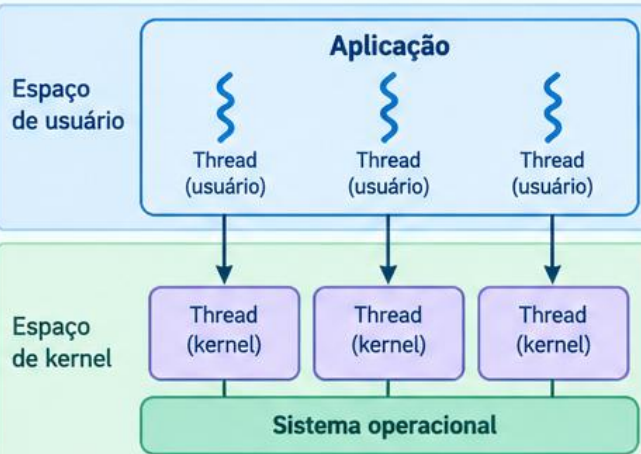
O gerenciamento das threads é feito em espaço de usuário, pela própria aplicação.



- Criação, escalonamento e sincronização feitos pela biblioteca de threads (runtime).
- O kernel vê apenas o processo (um thread).
- Mais leve e rápido na criação e troca de contexto.
- Não permite paralelismo real em multiprocessadores (salvo com múltiplos processos).

Kernel-Level Threads

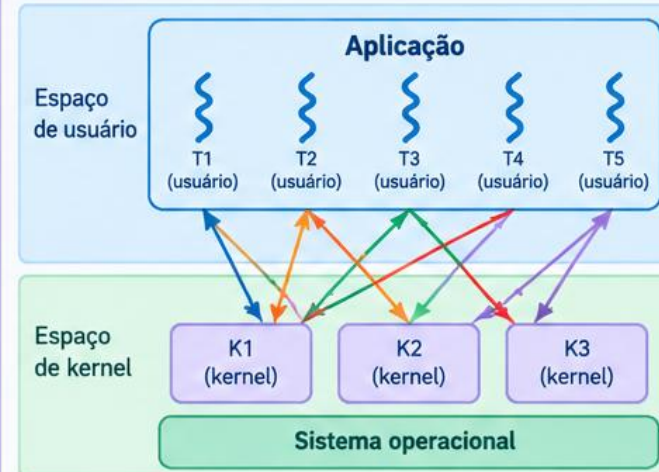
As threads são gerenciadas diretamente pelo sistema operacional (kernel).



- Cada thread de usuário é associada a uma thread do kernel.
- O kernel faz o escalonamento das threads.
- Permite paralelismo real em multiprocessadores.
- Criação e troca de contexto são mais custosas (chamadas ao kernel).

Many-to-Many

Muitas threads de usuário são mapeadas em um número igual ou menor de threads do kernel.



- Várias threads de usuário são mapeadas em um número menor (ou igual) de threads do kernel.
- O kernel faz o escalonamento das threads do kernel.
- Combina a leveza das threads de usuário com o paralelismo do kernel.
- Mais complexo de implementar.

Característica	User-Level	Kernel-Level	Many-to-Many
Gerenciamento das threads	Biblioteca de usuário	Sistema operacional (kernel)	Biblioteca de usuário + kernel
Visão do kernel	Vê apenas o processo	Vê todas as threads	Vê um número menor de threads do kernel
Criação e troca de contexto	Rápida (no espaço de usuário)	Mais lenta (chamada ao kernel)	Intermediária
Paralelismo em multiprocessadores	Não (salvo com múltiplos processos)	Sim	Sim
Complexidade de implementação	Mais simples	Mais simples (mas menos eficiente)	Mais complexa
Exemplo de uso	Bibliotecas como pthreads (em alguns SOs)	Sistemas que priorizam controle e segurança	Sistemas modernos (ex.: Windows, Solaris, Linux)

Threads em sistemas distribuídos

- **Chamadas bloqueadoras sem bloquear o processo:** uma thread bloqueia em `read()/recv()`, as demais continuam executando
- **Múltiplas conexões lógicas simultâneas:** uma thread por conexão/socket, alternativa mais simples de programar que multiplexação manual (`select/poll/epoll`)
- Essas duas ideias sustentam os padrões que veremos a seguir: clientes e servidores multithread

Ferramentas / comandos

```
$ netstat -tnp
```

Conexões TCP simultâneas de um processo servidor

```
$ ss -tnp
```

Equivalente moderno, mais rápido

Cliente Multithread

Um **cliente multithread** é uma aplicação cliente (isto é, que consome serviços de um servidor) **projetada para executar várias tarefas simultaneamente** usando múltiplas *threads* dentro de um mesmo processo.

1. **Uso de múltiplas threads de execução** dentro de um único processo cliente.
2. **Compartilhamento de recursos**, como memória, conexões de rede e dados em cache.
3. **Concorrência interna:** cada thread pode cuidar de uma subtarefa específica.
4. **Melhor desempenho e responsividade**, pois operações de E/S (como leitura de rede) não bloqueiam todo o cliente.



Cliente Multithread

Cenário

Threads no Cliente

Navegador Web

Uma thread renderiza a página, outra carrega imagens, outra lida com scripts.

Aplicativo de download

Uma thread por arquivo ou por segmento de arquivo sendo baixado.

Cliente de chat distribuído

Uma thread envia mensagens, outra recebe mensagens, outra atualiza a interface.

Cliente de banco distribuído

Uma thread cuida da comunicação segura, outra processa respostas do servidor.

Servidores Multithread

- Um **servidor multithread** é um tipo de servidor projetado para **atender múltiplos clientes simultaneamente**, usando **várias threads de execução** dentro de um mesmo processo.
- Em vez de criar um processo separado para cada cliente (como em um servidor *multiprocess*), o servidor **cria uma nova thread** para cada requisição — ou reutiliza threads de um *pool* —, o que torna o atendimento **mais eficiente e escalável**.

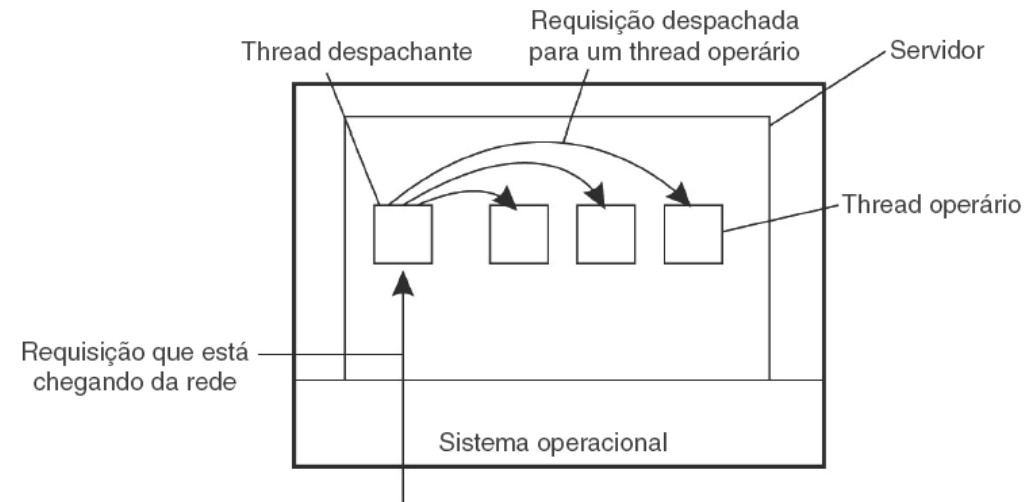


Figura 3.3 Servidor multithread organizado segundo modelo despachante/operário.

Servidores Multithread

- **Funcionamento Básico**

- O servidor principal (*thread mestre*) **ouve** conexões em uma porta de rede (ex: TCP).
- Quando um cliente se conecta, o servidor:
 - Cria uma **nova thread** para atender aquele cliente, ou
 - Atribui o cliente a uma thread livre de um **pool de threads**.
- Cada thread lida com a comunicação (envio e recebimento de mensagens) **de forma independente**.
- Quando o atendimento termina, a thread é encerrada ou devolvida ao pool.

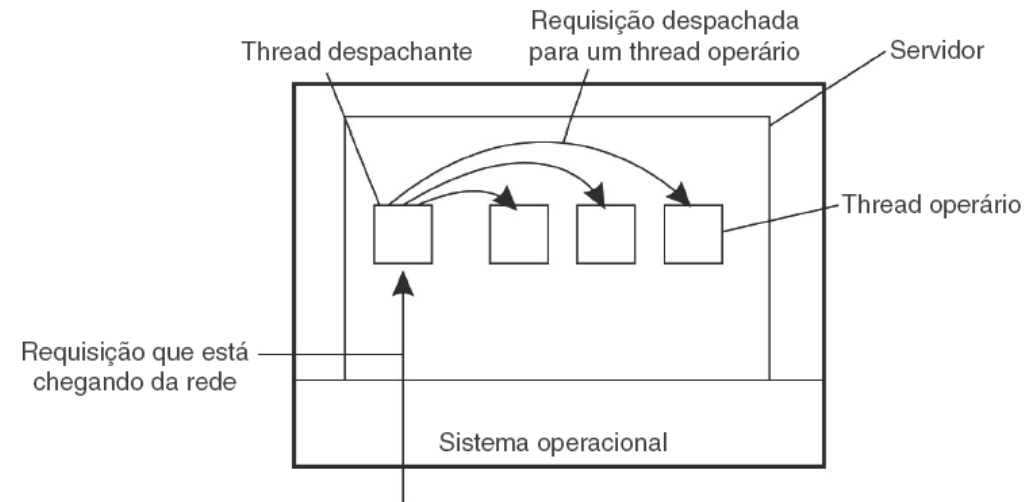


Figura 3.3 Servidor multithread organizado segundo modelo despachante/operário.

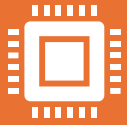
Servidores Multithread

Categoria	Benefício
Desempenho	Criação e troca de contexto entre threads é muito mais leve que entre processos.
Escalabilidade	Pode atender centenas ou milhares de clientes simultaneamente.
Responsividade	Um cliente lento não bloqueia os outros.
Compartilhamento de Dados	Threads podem acessar dados comuns (cache, sessão, logs) com facilidade.

Virtualização e Clientes

Tipos e arquiteturas de VM, interfaces de rede, X Window, thin clients e software do lado cliente

Virtualização



Virtualização: técnica que permite executar múltiplos ambientes isolados sobre o mesmo hardware físico.



Nos sistemas distribuídos, é usada para abstrair, particionar e gerenciar recursos de forma flexível e escalável.



Desacopla hardware físico de sistemas operacionais e aplicações.

Ferramentas / comandos

```
$ lscpu
```

Recursos de CPU 'reais' do host

```
$ free -h
```

Memória física vs. o que uma VM expõe

```
$ lsblk
```

Dispositivos de bloco disponíveis

Papel da Virtualização

Abstração de recursos.

Isolamento entre aplicações.

Escalabilidade e elasticidade.

Migração e tolerância a falhas.

Gerenciamento centralizado via orquestradores
(Kubernetes, OpenStack).

Benefícios



Flexibilidade: múltiplos SOs no mesmo hardware.



Eficiência: melhor uso de CPU, memória e rede.



Disponibilidade: replicação e recuperação rápidas.



Manutenção: clonagem e migração de instâncias.



Segurança: isolamento entre ambientes.

Arquiteturas de Máquinas Virtuais

Nível de interface	O que é virtualizado	Exemplo
Conjunto de instruções (ISA)	Hardware completo, inclusive arquitetura diferente	QEMU emulando outra arquitetura
Chamadas de biblioteca (API)	Bibliotecas de sistema	WINE — compatibilidade Windows/Linux
Chamadas de sistema (ABI/syscall)	Interceptação de syscalls por hipervisor	KVM, VMware, VirtualBox
Aplicação	Máquina virtual de linguagem — bytecode portátil	JVM (Java), CLR (.NET)

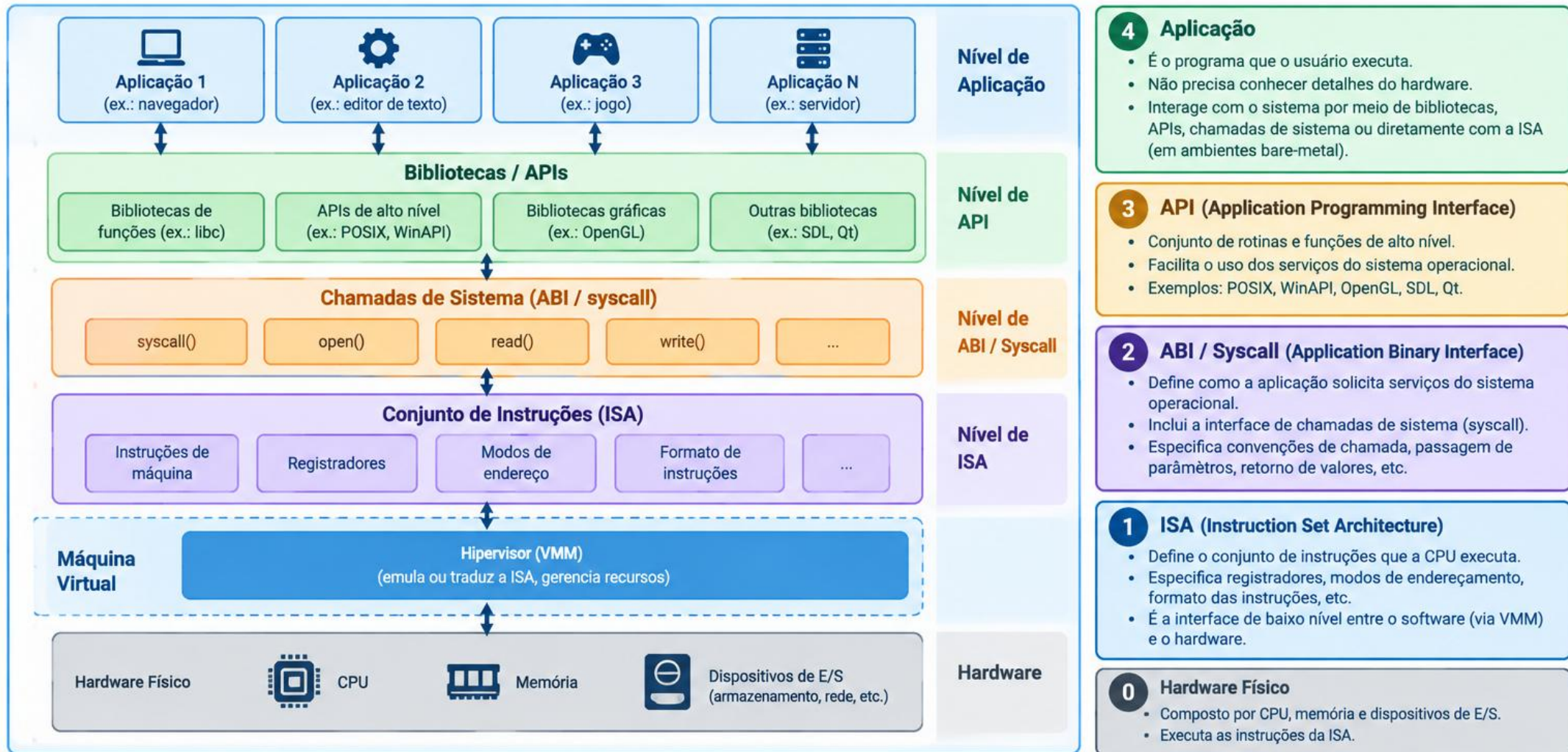
Hipervisor tipo 1 (bare-metal: Xen, KVM) vs. tipo 2 (hospedado: VirtualBox, VMware Workstation)

Ferramentas

\$ `virt-host-validate` · `docker info`
Suporte a KVM · virtualização em nível de SO (cgroups/namespaces) por contraste

Arquiteturas de Máquinas Virtuais – Níveis de Interface

Da aplicação ao hardware: diferentes níveis de abstração, diferentes interfaces.



1. ISA – Conjunto de Instruções

Interface entre o software e o hardware.
Define o que a CPU pode fazer.

Ex.: x86, ARM, RISC-V

2. ABI / Syscall – Chamadas de Sistema

Interface entre a aplicação e o sistema operacional.
Permite solicitar serviços do SO.

Ex.: syscall(), open(), read(), write()

3. API – Bibliotecas

Interface de alto nível para o programador.
Simplifica o uso dos serviços do sistema.

Ex.: libc, POSIX, WinAPI, OpenGL

4. Aplicação

Programa que o usuário executa.
Utiliza APIs ou chamadas de sistema para acessar os recursos.

Ex.: navegador, editor, jogo, servidor

Tipos de Virtualização

Virtualização de Hardware – cria máquinas virtuais completas (VMware, KVM, Hyper-V).

Virtualização de SO – containers isolados dentro do mesmo kernel (Docker, LXC).

Virtualização de Rede – redes virtuais sobre redes físicas (VLAN, SDN).

Virtualização de Armazenamento – combina ou particiona volumes (RAID, Ceph).

Virtualização de Aplicação – ambientes isolados para execução (JVM, WebAssembly).

Organização Básica

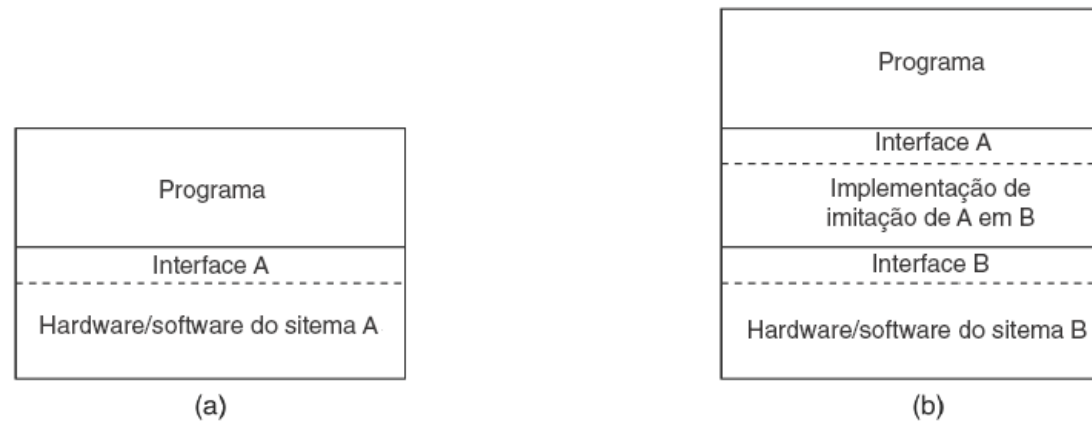


Figura 3.4 (a) Organização geral entre programa, interface e sistema. (b) Organização geral da virtualização do sistema A sobre o sistema B.

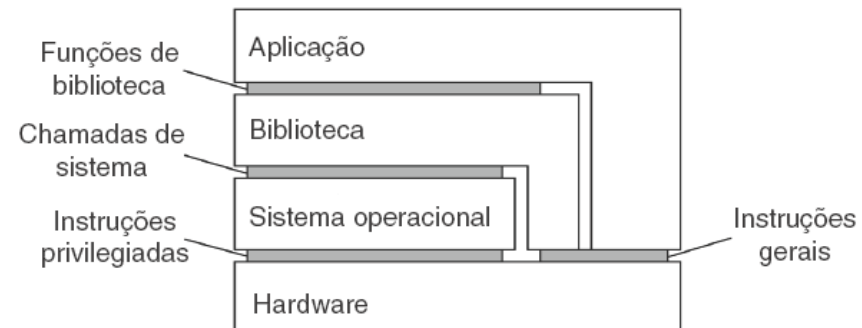
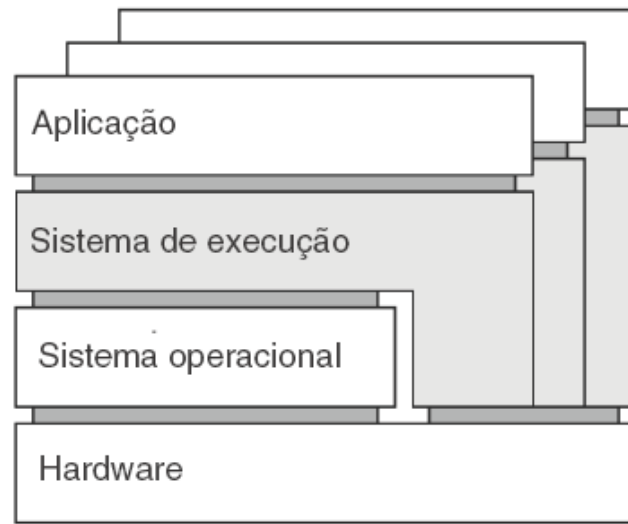
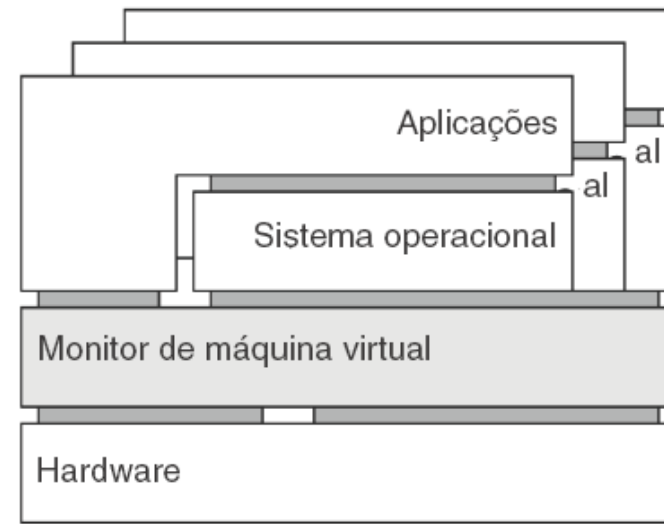


Figura 3.5 Várias interfaces oferecidas por sistemas de computadores.

Arquiteturas de Máquinas Virtuais



(a)



(b)

Figura 3.6 (a) Máquina virtual de processo, com várias instâncias de combinações (aplicação, execução).
(b) Monitor de máquina virtual com várias instâncias de combinações (aplicações, sistema operacional).

Interfaces de Usuários em Rede

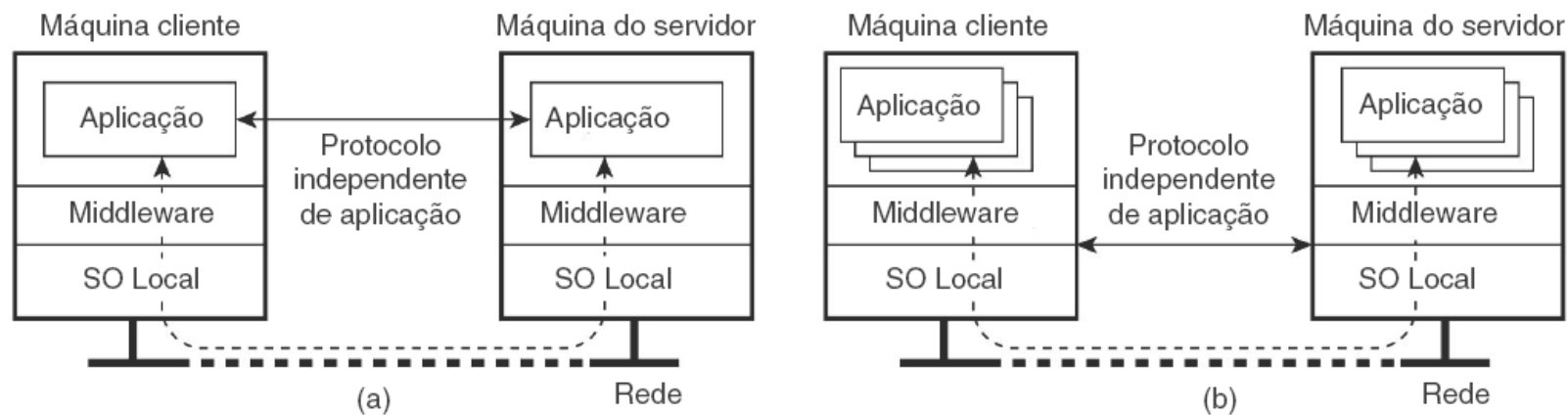


Figura 3.7 (a) Aplicação em rede com seu próprio protocolo. (b) Solução geral para permitir acesso a aplicações remotas.

Exemplo: Sistema X Window

- Comandos são enviados pela rede e na sequência são executados pelos núcleos X
- Aplicações escritas para X devem separar a lógica da aplicação dos comandos de interface de usuário

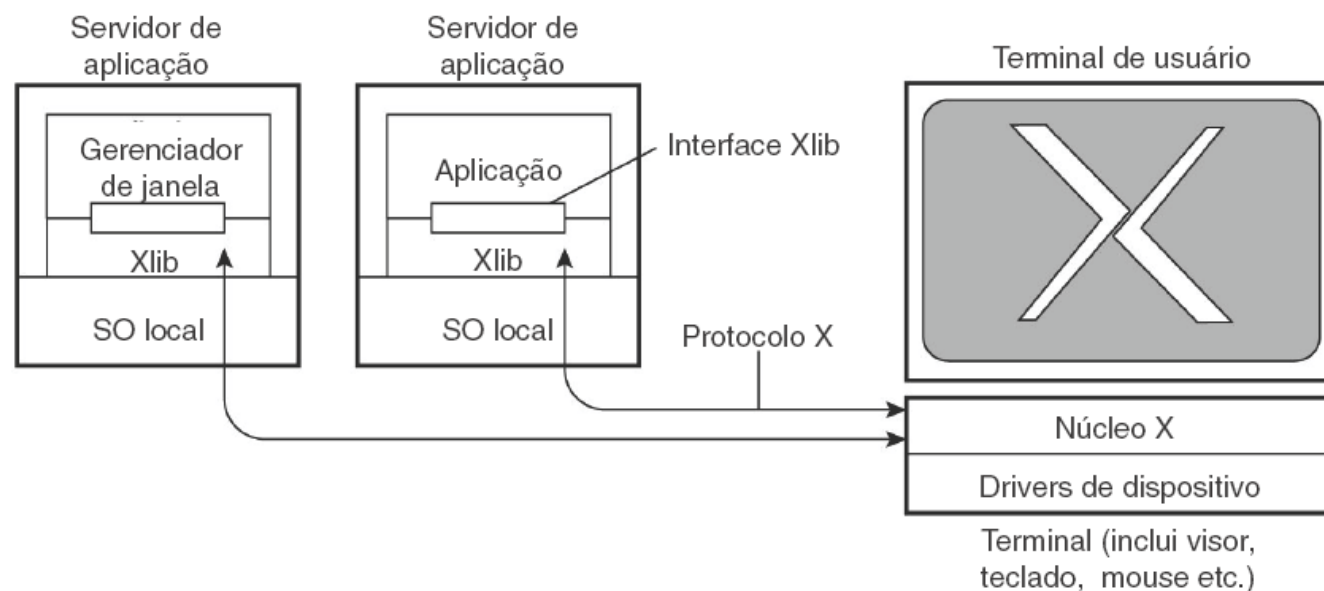


Figura 3.8 Organização básica do sistema X Window.

Software do lado cliente para transparência de distribuição

Partes do nível de processamento e dados são executadas do lado do cliente

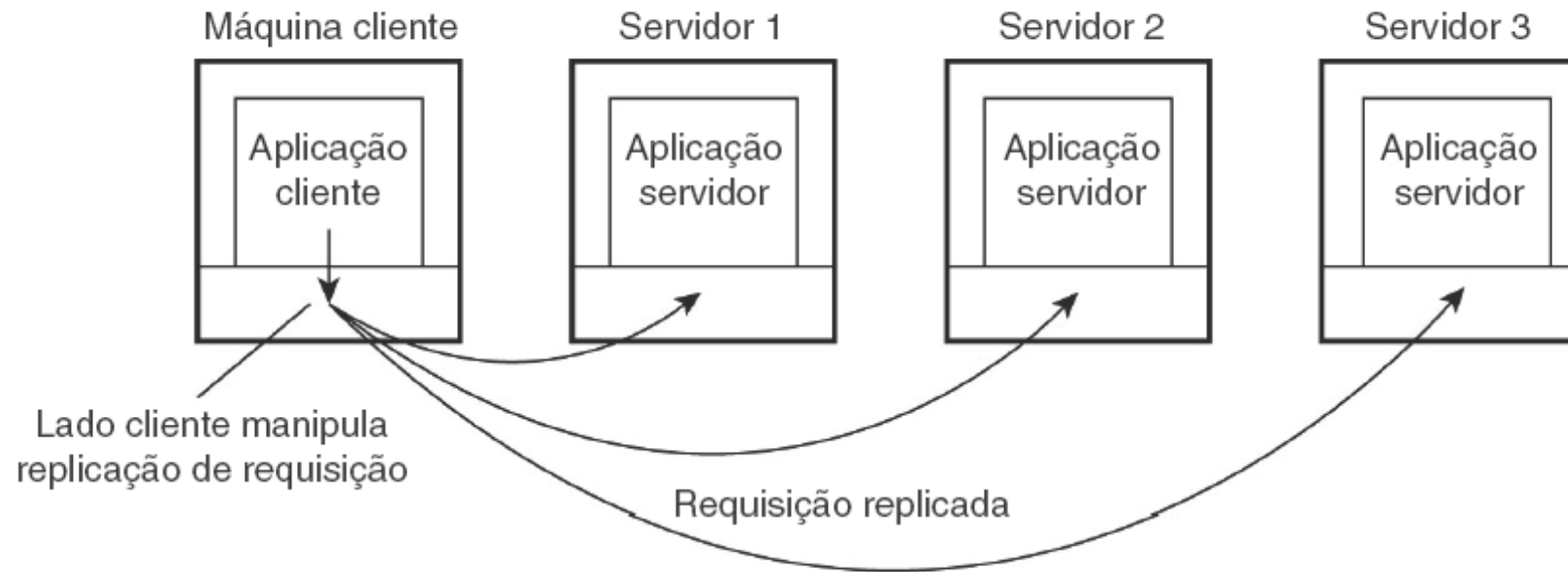


Figura 3.9 Replicação transparente de um servidor usando uma solução do lado cliente.

Servidores e Clusters

Tipos de servidor, estado, clusters, comutadores de camada de transporte e gerenciamento

Servidores

Definição Formal

Em um sistema distribuído, **servidor** é um **processo que gerencia um recurso compartilhado e responde a requisições** de outros processos (clientes) segundo um protocolo bem definido, geralmente em um modelo **cliente-servidor**.

Servidor interativo: manipula a requisição e retorna ao cliente.

Servidor concorrente: passa para um thread ou processo separado.

Portas conhecidas e exemplos

Serviço	Porta
FTP (controle)	TCP 21 (dados: TCP 20 em modo ativo)
HTTP	TCP 80 (HTTPS: TCP 443)

Exemplo didático: um servidor que apenas informa a hora corrente ilustra o modelo requisição-resposta mínimo.

Ferramentas / comandos

```
$ curl -v http://host
```

Observa o handshake HTTP

```
$ nc -l 21
```

Simula um socket de escuta simples

```
$ netstat -tlnp
```

Lista portas em escuta na máquina

Servidores

Ferramentas / comandos

```
$ curl -v http://host
```

Observa o handshake HTTP

```
$ nc -l 21
```

Simula um socket de escuta simples

```
$ netstat -tlnp
```

Lista portas em escuta na máquina

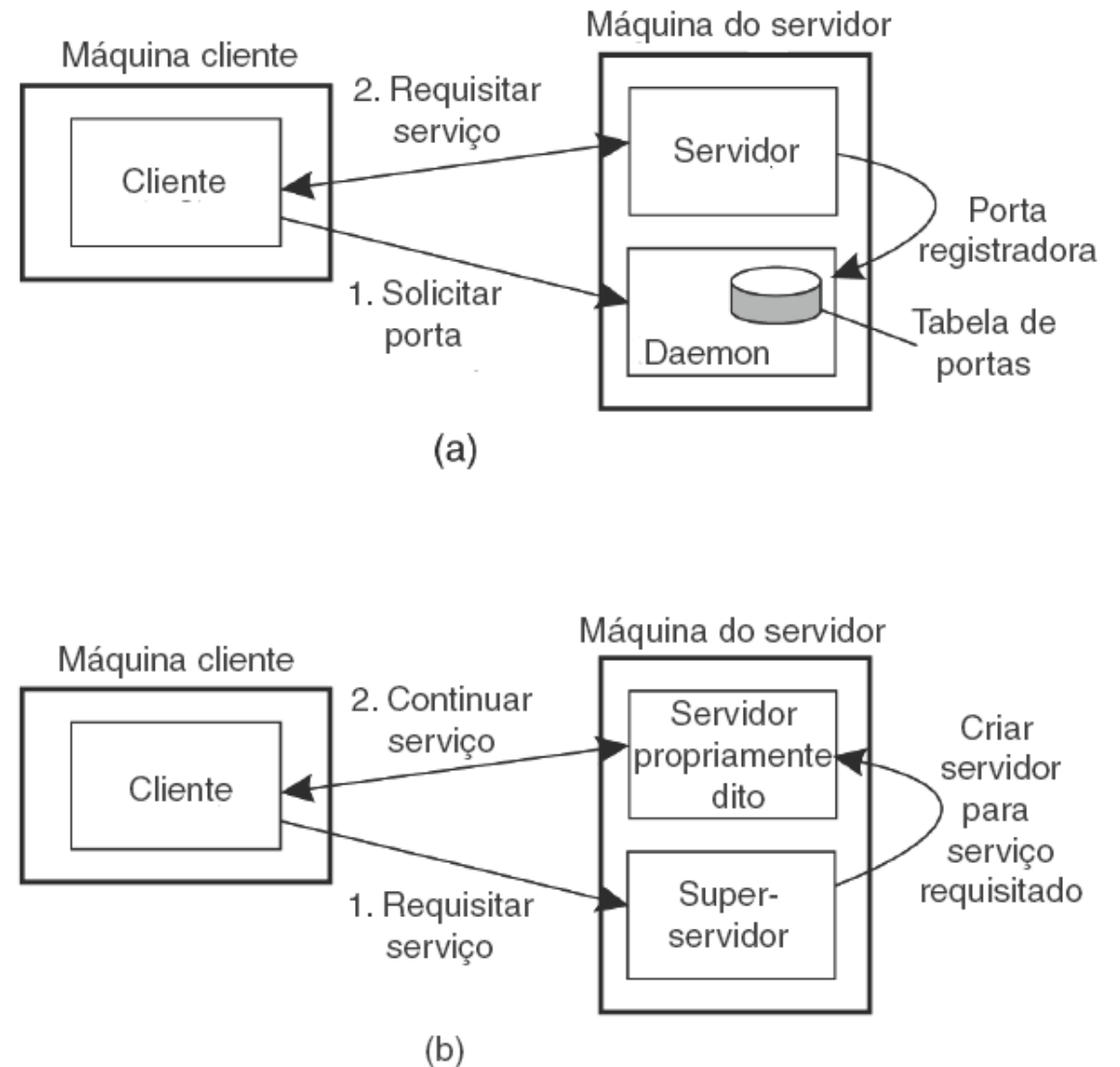


Figura 3.10 (a) Vinculação cliente-servidor usando um daemon. (b) Vinculação cliente-servidor usando um supervisor.

Características Principais

Característica	Descrição
Passividade	O servidor espera requisições ; ele não inicia comunicações por conta própria.
Persistência	Pode manter estado (dados, sessões, logs) entre requisições.
Confiabilidade	Deve garantir disponibilidade e consistência das respostas.
Escalabilidade	Pode atender múltiplos clientes simultaneamente (ex: via <i>multithreading</i> ou <i>load balancing</i>).
Transparência	Idealmente, o cliente não percebe se o servidor está local ou remoto.

Servidores

Um servidor sem estado não mantém informações sobre o estado de seus clientes e pode mudar seu próprio estado.

Um servidor com estado flexível promete manter estado em nome do cliente, mas apenas por tempo limitado

Já um servidor com estado mantém informações persistentes sobre seus clientes, por exemplo, servidor de arquivos com cópias no cliente

Existe diferença entre estado de sessão (temporário) e estado permanente

Estado do servidor

Tipo	Comportamento
Sem estado (stateless)	Não guarda nenhuma informação de requisições anteriores; cada requisição é autocontida (ex.: HTTP clássico)
Com estado flexível (soft state)	Promete manter estado em nome do cliente, mas só por tempo limitado — exige renovação (leases, TTLs)
Com estado (stateful)	Mantém informações persistentes sobre os clientes — ex.: servidor de arquivos com cópias em cache no cliente

Diferença crucial: estado de sessão (temporário, dura a interação) vs. estado permanente (dados duradouros do cliente). Servidores stateless escalam horizontalmente com mais facilidade — qualquer réplica atende qualquer requisição.

Tipos Comuns de Servidores



Tipo	Função
Servidor de Arquivos	Armazena e gerencia arquivos compartilhados (ex: NFS, Samba).
Servidor de Banco de Dados	Armazena dados estruturados e responde a consultas SQL (ex: PostgreSQL, MySQL).
Servidor de Aplicação	Executa lógica de negócio e processa requisições de clientes (ex: Apache Tomcat, JBoss).
Servidor Web	Fornece conteúdo HTTP/HTTPS para navegadores (ex: Apache, Nginx).
Servidor de Diretório	Mantém informações de autenticação e autorização (ex: LDAP).
Servidor de Nomes	Tradução de nomes lógicos em endereços físicos (ex: DNS).

Clusters de servidores

- Arquitetura geral: conjunto de máquinas (nós) atuando como um único serviço lógico, geralmente atrás de um front-end
- **Objetivo duplo:** escalabilidade horizontal (mais nós, mais capacidade) e tolerância a falhas (um nó cai, outros continuam)

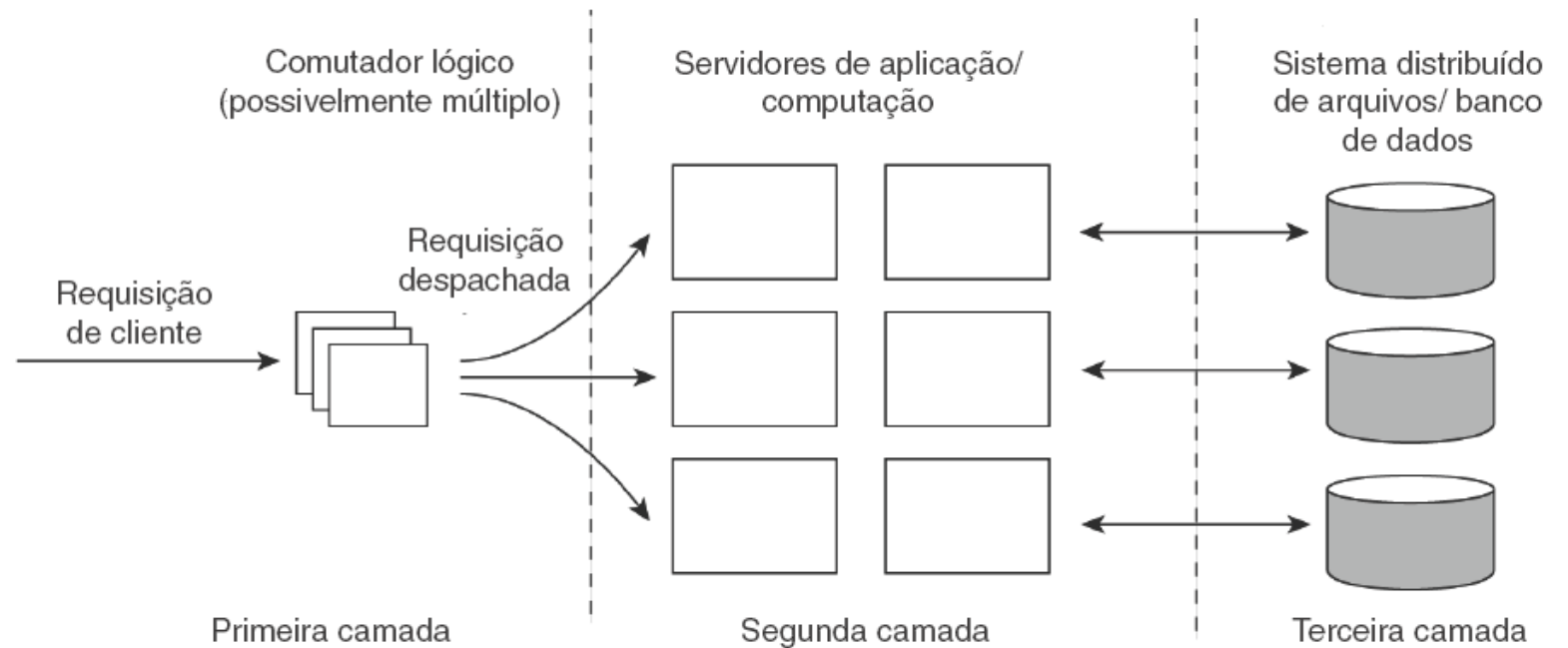
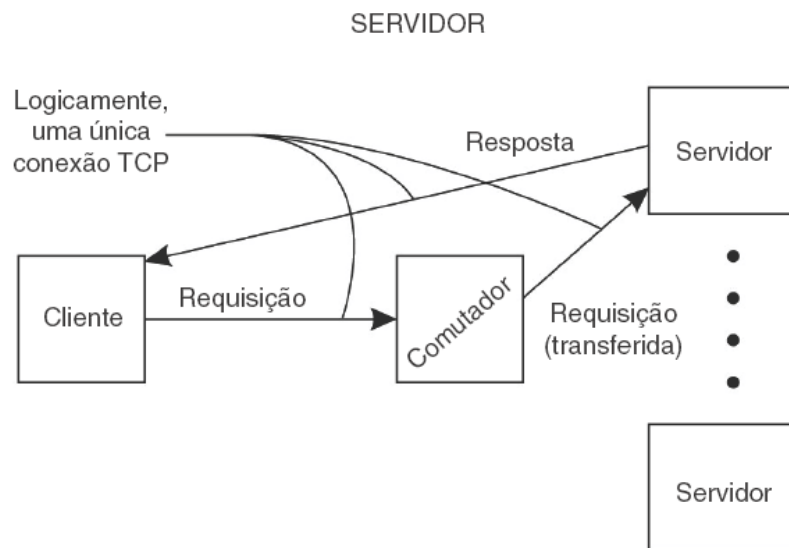


Figura 3.11 Organização geral de um cluster de servidores de três camadas.

Comutadores da camada de transporte

- Recebem as conexões TCP/UDP externas e as distribuem entre os servidores reais do cluster
- Mantêm a ilusão de um único endereço e porta externos para o cliente
- **Técnicas de distribuição:** round-robin, least-connections, hashing por IP de origem (para afinidade de sessão)



Ferramentas de referência

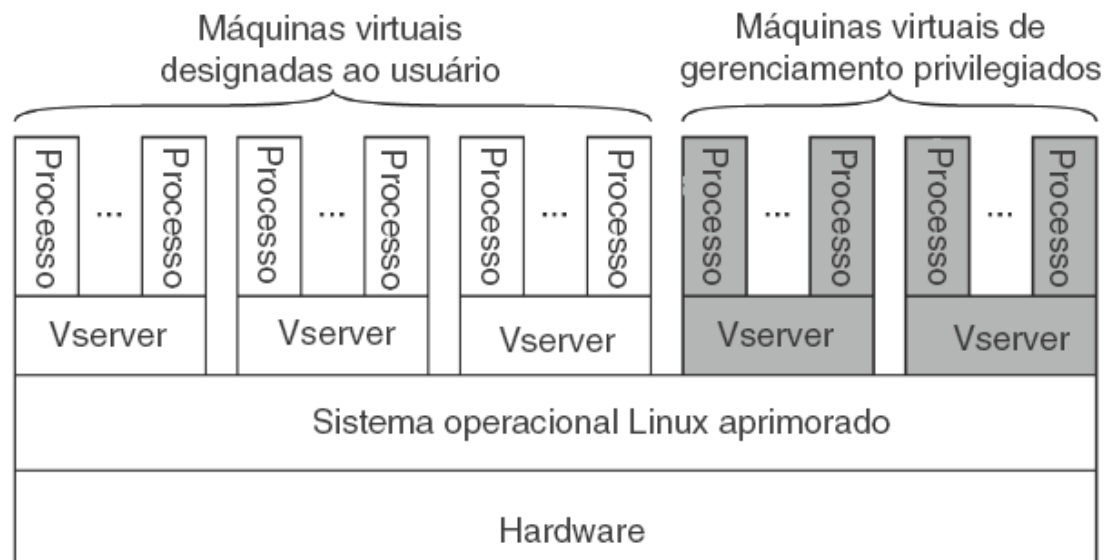
```
$ haproxy -f haproxy.cfg  
Load balancer / reverse proxy
```

```
$ nginx -t && nginx -s reload  
NGINX como reverse proxy
```

```
$ ipvsadm -L -n  
Regras de balanceamento do LVS/IPVS
```

Gerenciamento de clusters de servidores

- Monitoramento de saúde dos nós (health checks) e adição/remoção dinâmica de nós
- Sincronização de estado entre réplicas, quando aplicável
- Evolução moderna do conceito descrito no material: orquestradores de contêineres



Ferramentas / comandos

```
$ kubectl get nodes
```

Estado dos nós de um cluster Kubernetes

```
$ kubectl get pods -o wide
```

Distribuição dos processos pelos nós

```
$ docker service ls
```

Equivalente em Docker Swarm

Servidores distribuídos

- O 'servidor' lógico do ponto de vista do cliente é, na prática, um conjunto de servidores físicos coordenados
- Reforça, do lado do servidor, a mesma ideia de transparência de distribuição vista do lado do cliente

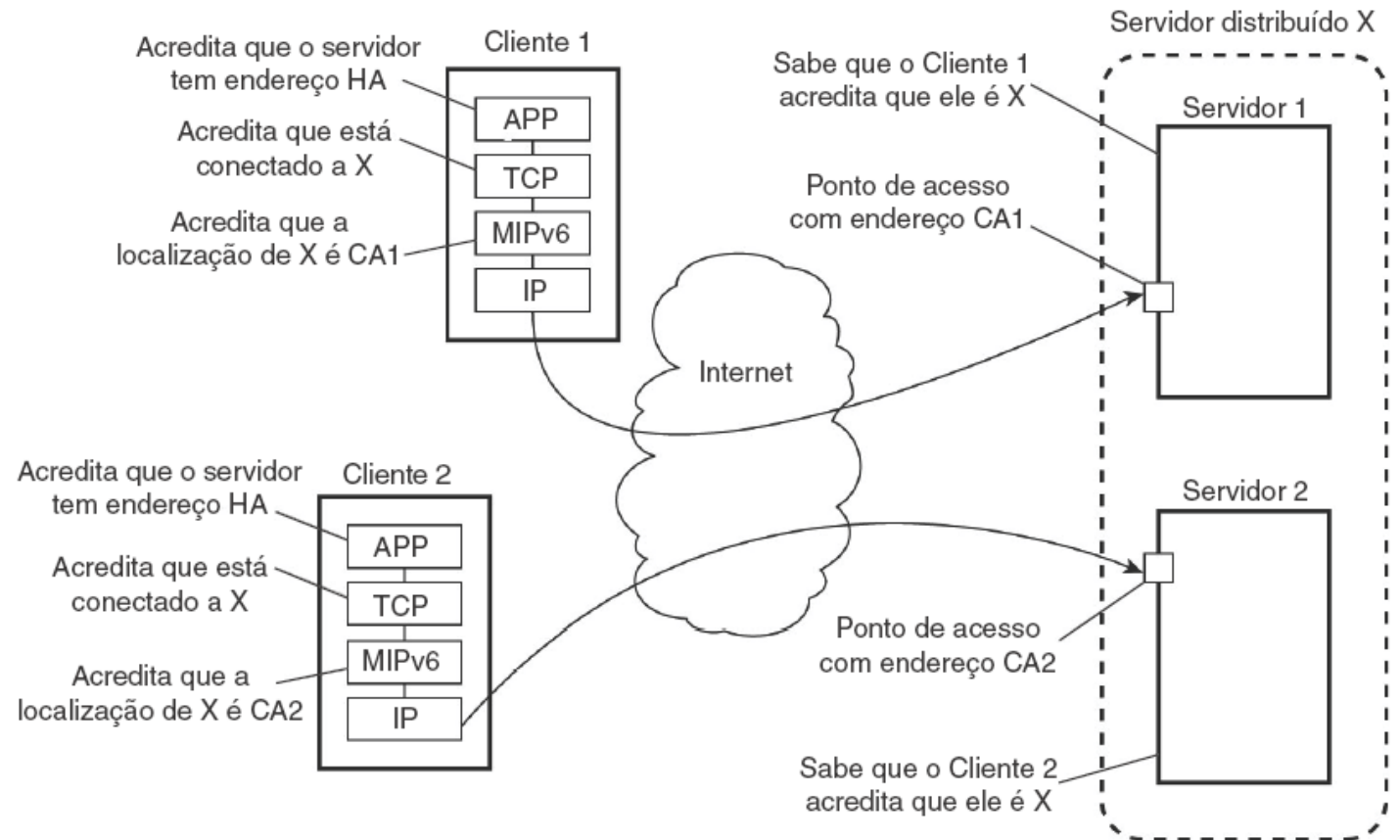


Figura 3.13 Otimização de rota em um servidor distribuído.

Migração de Código

Motivação, modelos de mobilidade e vinculação recurso-máquina

Motivação para migração de código

- **Balanceamento de carga:** mover processamento para onde há recursos ociosos
- **Redução de latência:** mover código para perto dos dados que ele processa, quando a computação é mais leve de transportar que o dataset
- Reconfiguração dinâmica de sistemas sem interrupção de serviço
- **Segurança e isolamento:** código migrante executa em ambiente sandboxed

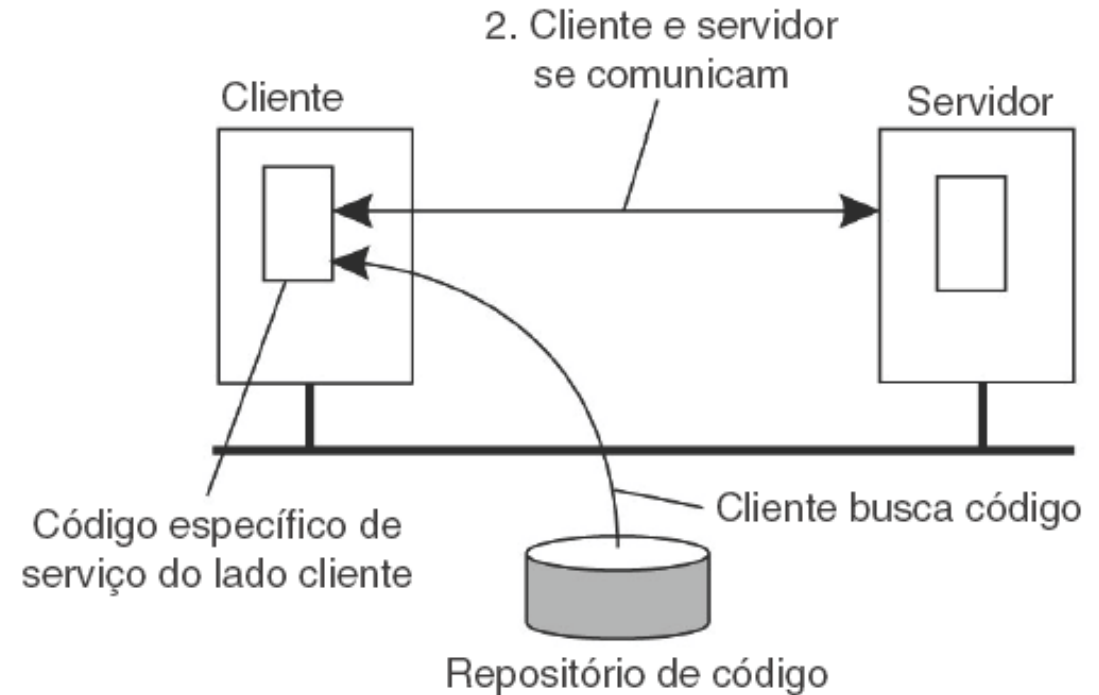


Figura 3.16 Princípio de configuração dinâmica de um cliente para se comunicar com um servidor. Primeiro o cliente busca o software necessário e então invoca o servidor.

Modelos de migração de código

Modelo	O que migra	Exemplo
Migração fraca (weak mobility)	Apenas o código; a execução recomeça do início no destino	Applet Java carregado do zero em nova JVM
Migração forte (strong mobility)	Código e estado de execução (pilha, PC) juntos — retoma de onde parou	Sistemas de agentes móveis (Telescript, Aglets)

Outra distinção: migração sender-initiated (a origem decide enviar) vs. receiver-initiated (o destino solicita o código).

Migração de Código

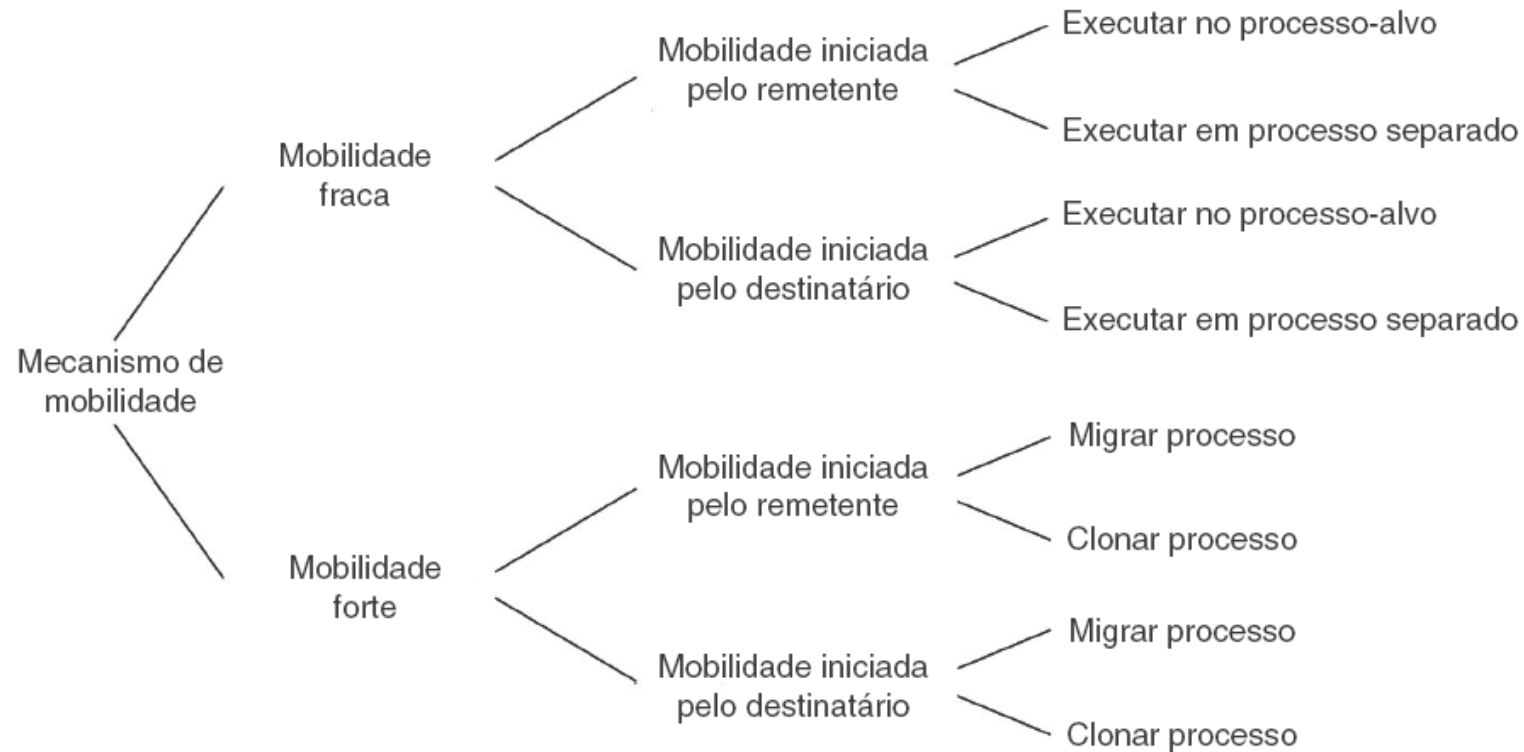


Figura 3.17 Alternativas para migração de código.

Requisitos de infraestrutura para migração

- Formato de código portátil entre arquiteturas heterogêneas — conecta com o nível de aplicação da virtualização (bytecode JVM/.NET)
- Mecanismos de segurança para executar código de origem não confiável — sandboxing, permissões restritas

Migração de código hoje

```
$ docker commit / save / load
```

Empacota o estado de um contêiner para mover a outra máquina

```
$ criu dump -t <pid>
```

Checkpoint/Restore in Userspace — migração forte real de processos Linux

Conclusão

Seção 1	Threads	A unidade básica de execução concorrente — mais barata que um processo porque compartilha memória, mas ainda sujeita ao custo real do chaveamento de contexto
Seção 2	Virtualização e Clientes	O ambiente isolado e portátil onde as threads executam — e os mesmos papéis, do lado do cliente, que threads e IPC tornam possíveis
Seção 3	Servidores e Clusters	O mesmo papel, agora do lado do servidor — e como multiplicá-lo em clusters para ganhar escala horizontal e tolerância a falhas
Seção 4	Migração de Código	A capacidade de mover essas unidades de execução entre ambientes virtualizados — fechando o ciclo entre todos os conceitos anteriores

Do processador virtual ao cluster distribuído, a mesma ideia se repete em cada camada: threads e processos são as unidades de execução que sistemas distribuídos movem, isolam e coordenam.

Obrigado!

Perguntas e discussão

Prof. Paulo Bressan

Universidade Federal de Alfenas — UNIFAL-MG